

EagleEye: Intelligent Edge AI Surveillance

Daily Change Report and Development History

Project repository: `fyp-eagle-eye`

Report revised: 19 May 2026

Synced through the 96×96 RGB MobileNetV1 0.2 sketchboard deployment and AI-assisted hard-negative capture update

Abstract

This document records project evolution in a professional, date-ordered format. Entries follow `git log` on `main`. The May 2026 additions now cover the model-training layout, hard-negative collection, knowledge distillation experiments, Edge Impulse grayscale/RGB testing, and the firmware/runtime lessons learned while moving from lab training to ESP32-CAM validation.

Contents

1	Purpose and conventions	2
2	Chronological development summary	2
2.1	25 October 2025	2
2.2	21 November 2025	2
2.3	26 November 2025	2
2.4	8 December 2025	2
2.5	10 December 2025	2
2.6	14 December 2025	2
2.7	10 February 2026	2
2.8	14 February 2026	2
2.9	16 February 2026	3
2.10	17 February 2026	3
2.11	18 February 2026	3
2.12	6 March 2026	3
2.13	21 April 2026	3
2.14	2 May 2026 (7dc9cc93)	3
2.15	3 May 2026 (3f6989fd)	3
2.16	6 May 2026 (0c41c75e)	3
2.17	8–9 May 2026	4
2.18	9–10 May 2026	4
2.19	11 May 2026	4
2.20	16 May 2026 (60b59617)	4
2.21	17 May 2026 (a07039ba)	4
2.22	19 May 2026	5
2.23	7–9 June 2026	6
3	Suggested maintenance	6
4	Compilation	6

1 Purpose and conventions

- **Scope:** EagleEye spans ESP32-CAM firmware, a Python MQTT–cloud bridge, a React Native (Expo) mobile client, model training assets, and documentation.
- **Sources:** Commit subjects and bodies are paraphrased for clarity; hashes identify the corresponding revision where applicable.
- **Updates:** Append new dated subsections when material changes land in `main`. Recompile this `.tex` file to refresh the PDF.

2 Chronological development summary

2.1 25 October 2025

Milestone: repository bootstrap. Initial README and project upload; early ESP32-CAM bring-up on university Wi-Fi infrastructure.

2.2 21 November 2025

Milestone: on-device inference. TensorFlow Lite pipeline validated; interactive verification via DumbDisplay (tap-to-classify prototype).

2.3 26 November 2025

Maintenance. Minor commit (no extended message in history).

2.4 8 December 2025

End-to-end IoT path. Human detection with approximately 712 ms latency; intruder JPEG relayed via MQTT; Cloudinary storage with metadata URLs persisted in Firebase Realtime Database. Modular layout: camera sketch with `EagleEye_IoT.h` for connectivity.

Security hygiene. `.gitignore` extended to exclude secrets (`.env`, `secrets.h`, `serviceAccountKey.json`).

2.5 10 December 2025

Documentation. README expanded to describe system architecture and usage.

2.6 14 December 2025

Local MQTT and payload efficiency. Mosquitto-based local broker; bridge subscribed for diagnostics; ESP32 transmits raw binary JPEG (larger MQTT buffer, tuned thresholds and cooldown). Supporting assets: Mosquitto configuration, training notebooks, quality simulation utilities, dataset maintenance.

2.7 10 February 2026

Maintenance. Minor commit.

2.8 14 February 2026

Custom tiny model family. In-house training artifacts under a dedicated model directory; reference to a ~ 106 ms variant and Arduino export layout.

2.9 16 February 2026

Greyscale TinyML deployment (46847c76). INT8 CNN at $48 \times 48 \times 1$; RGB565 capture with software greyscale for inference; color evidence uploads unchanged; tensor arena reduced to 100 kB; simplified capture path in `EagleEye_IoT.h`.

2.10 17 February 2026

Command and control (f803947c). Mobile: tabbed navigation, dashboard with arm/disarm and latest alert preview, heartbeat-driven online/offline badge. Bridge: Firebase listener on `config/armed`, intrusion gating when disarmed, periodic `status/heartbeat` updates. Manual start guide updated for USB/ADB workflow.

2.11 18 February 2026

Imaging pipeline (c79c10f7, 719dae9b). QVGA capture with center 240×240 crop prior to model resize—undistorted “digital zoom” for improved range; serial diagnostics updated.

2.12 6 March 2026

Academic milestone. Mid FYP presentation completed (status commit).

2.13 21 April 2026

Repository restructure (179ef260). Consolidated layout: `firmware/eagleeye-main`, `backend`, `mobile-app`, `model-training`, `datasets`, `docs`, `_archive`; `sketch` renamed to `eagleeye-main.ino`; manuals and README paths corrected.

Power-aware sensing (d87859da). PIR on GPIO 14, deep sleep, timed AI windows, armed-state respect, camera/Wi-Fi de-init before sleep; SD disabled to free the PIR pin.

2.14 2 May 2026 (7dc9cc93)

Live preview (first pass). Git message is short; the merge introduced HTTP streaming from the camera (`esp_http_server`, MJPEG at / on port 80), integration in `eagleeye-main.ino`, initial `LiveMonitorScreen.js` (WebView-based), dashboard navigation to Live, `pir_debug` test sketch, minor `bridge.py` edits, and test images under `backend/captures/`.

2.15 3 May 2026 (3f6989fd)

Live preview reliability + gallery bulk delete. Firmware: `eagleeye_grab_jpeg()` and `GET /capture`; WebSocket binary JPEG stream on port 81 (`eagleeye_ws.h`, WebSockets library); latch reset when preview ends so AI alerts work again. Mobile: WebSocket + `expo-image` live path, Android uses `CleartextTraffic`, remove unused `react-native-webview`; **Delete all** alerts with Cloudinary queue via `deletion_requests`. Documentation: this `docs/daily_changelog/tree`.

2.16 6 May 2026 (0c41c75e)

Model-training hygiene (refactor(model-training)). Experimental and superseded Python training scripts were moved into `model-training/_archive/` (e.g. COCO/Colab helpers, colour-model variants, transfer-learning and verification utilities) so the active tree is not cluttered. The single **production** training entry point for the deployed greyscale TinyML model is consolidated under `model-training/workable_model/train_tiny_model.py` (48×48 greyscale, ESP32-oriented). A new `sketchboard/` tree provides a **scratch workspace**: copies of the

working trainer, a place for firmware experiments, `notes/`, and a README describing safe day-to-day experimentation without editing production paths.

2.17 8–9 May 2026

ESP32-CAM Hard Negative Capturer. Developed a local Python Flask server (`firmware/tools/hard_neg`) paired with a custom ESP32 sketch to manually capture false positives/negatives. Images are POSTed over Wi-Fi and sorted into `Humans/NonHuman` datasets. Addressed system reboots (EV-VSYNC-OVF) and Watchdog Timer timeouts during inference.

2.18 9–10 May 2026

ML Architecture Evaluation & Presentation Prep. Prepared to defend the Kaggle "Overthinkers" submission for CodeRush '26. Focused on validating the multi-model bagging/boosting pipeline, Optuna hyperparameter optimization, and adaptive blending logic.

2.19 11 May 2026

Hard Negative Retraining. Fine-tuned the custom tiny model in `sketchboard/` to integrate hard negative datasets, targeting 90%+ accuracy. Added data augmentation (flip, rotate, zoom) in `train_in_sketchboard.py`. Exported updated INT8 `.tflite` models and generated C-headers (`human_detect_model_data_augmented.h`) for firmware integration.

2.20 16 May 2026 (60b59617)

Distillation and validation pass. The work moved beyond the earlier single hard-negative retraining loop and tested a more systematic model-improvement path. Fresh ESP32-CAM validation captures were added under `sketchboard/dataset/`, the exported model header was refreshed in `sketchboard/human_detect_model_data.h`, and the training/evaluation scripts under `sketchboard/knowledge_distillation/` were used to compare a larger teacher model with a smaller ESP32-friendly student model.

Changed files. Key files for this day include `sketchboard/dataset/*20260516*.jpg`, `sketchboard/knowledge_distillation/train_teacher.py`, `train_distillation.py`, `train_kd_optuna.py`, `evaluate_tflite.py`, `sketchboard/train_optuna_cv.py`, `sketchboard/sketchboard_hard_negative_90p`, and the generated model header used by sketchboard firmware tests.

New method. Instead of only adding more hard-negative images and retraining the same tiny network, this pass used a teacher/student knowledge distillation workflow. Optuna was introduced to search for better distillation settings such as learning rate, dropout, temperature, and alpha, making the model selection process more explainable than manual tuning.

Result / lesson. The experiment showed a practical direction for improving accuracy without forgetting the ESP32-CAM constraint: the student model still needs to stay small and fast, but it can learn from a stronger teacher and be judged with repeatable metrics rather than only live ad-hoc tests.

2.21 17 May 2026 (a07039ba)

Hard-negative capture expansion and Edge Impulse deployment test. This day focused on collecting a larger real-camera dataset and checking which model export path was actually usable on the ESP32-CAM. New human and non-human captures were added under `sketchboard/dataset/Humans/` and `sketchboard/dataset/NonHuman/`, while older tracked capture files from 9–13 May were cleaned out where they had been superseded by the newer organized run.

Changed files. Important files include `sketchboard/dataset/*20260517*.jpg`, `sketchboard/firmware/collect_hard_negatives.py`, `simple_server.py`, `firmware/tools/hard_negative_capturer/`, `models/MODEL_VERSIONS.md`, `models/tflite_to_cpp_header.py`, and archived model headers such as `model_v1.0_baseline.h`, `model_v6.0_edge_impulse_final.h`, and `model_v6.1_edge_impulse_gray`.

New method. The comparison changed from “does the model classify this image?” to “which deployment path behaves correctly on hardware?” The older Edge Impulse RGB export expected $48 \times 48 \times 3$ input, while the newer grayscale export used $48 \times 48 \times 1$, matching the firmware pipeline. The sketchboard firmware was simplified into a serial-only detector so testing could focus on clear `Human detected` / `Not detected` output and timing logs instead of the earlier Wi-Fi/web UI flow.

Firmware/runtime lesson. The ESP32 tests also exposed a runtime cost: broad `AllOpsResolver` usage can make inference unnecessarily heavy. The hard-negative capturer path therefore moved toward `MicroMutableOpResolver`, registering only the operations required by the selected model. The Flask capture server was made easier to debug with a `/test` endpoint, GET/POST handling on `/save`, and request-size logging so upload problems are visible during collection.

Documentation link. `README.md`, `PROJECT_MANUAL_START.md`, and `models/MODEL_VERSIONS.md` were updated around this workflow so the report, setup guide, and model archive describe the same RGB-vs-grayscale decision and the same hard-negative capture process.

2.22 19 May 2026

RGB MobileNetV1 0.2 model selection and ESP32-CAM runtime debugging. The project moved from the earlier 48×48 greyscale custom-tiny path toward a stronger 96×96 RGB transfer-learning pipeline for sketchboard experiments. The motivation was practical: the $\sim 5k$ -parameter custom CNN delivered low latency, but failed to generalize reliably in real camera scenes because the 48×48 greyscale crop removed too much human detail. The new candidate uses a pretrained MobileNetV1 backbone with width multiplier 0.2, a 32-neuron head, 0.25 dropout, class weighting, full image augmentation, 40 epochs, learning rate 0.00035, and batch size 16.

Validation result. The selected INT8 deployment reached 89.86% validation accuracy with a confusion matrix of $\begin{bmatrix} 59 & 12 \\ 3 & 74 \end{bmatrix}$ in label order `human`, `nonhuman`. This corresponds to approximately 95.2% human precision, 83.1% human recall, and 88.1% human F1. The result documents the core TinyML trade-off for EagleEye: the smallest possible model is not necessarily the best model; the deployed model must balance accuracy, latency, memory, and real-world reliability on the ESP32-CAM S1.

Firmware/runtime lesson. Side-by-side PC and device checks showed that the camera/preprocessing path and the inference engine must be validated together. Earlier ESP32-CAM tests exposed saturated incorrect predictions when ESP-NN acceleration was enabled for the RGB MobileNet path, while the non-accelerated TFLite Micro path produced correct results. The generated Arduino library was therefore packaged with ESP-NN disabled for correctness on this target.

Hard-negative capture update. `sketchboard/firmware/hard_negative_capturer/hard_negative_c` was converted from the old serial-only 48×48 greyscale TFLite sketch into a Wi-Fi AI-assisted hard-negative capturer using the installed `final_inferencing` library. It now serves a browser UI, streams JPEG frames over WebSocket, exposes `/ai` for live human/nonhuman scores, and posts labelled JPEGs to the sketchboard Python receiver at `/save`. This restores the same workflow as the original hard-negative capturer while testing the new 96×96 RGB MobileNetV1 model.

New/updated tooling. Added a repeatable training script, `tools/edge_impulse/retrain_rgb_96_mobil` and kept diagnostic helpers under `tools/diag/` for ESP32 frame dumps and PC/device inference

comparison. `models/MODEL_VERSIONS.md` was updated to record the architecture, metrics, and deployment trade-offs for the new sketchboard model.

2.23 7–9 June 2026

App-controlled pan head (two-board servo) and smoother live preview. This session added remote camera-pan control driven from the mobile app, and fixed the live-stream stutter and brownout resets observed while panning the camera.

Servo pan path (app → main → helper). A second ESP32-CAM acts as a dedicated *helper* (camera disabled) that drives one servo as a pan head. The mobile app sends an absolute angle to the main board over HTTP (`GET /servo?angle=N` on port 80, handled in `camera_web_server.h`); the main forwards it to the helper over a one-wire UART link (main GPIO15 TX → helper GPIO13 RX, common ground) as `ANGLE:N`. The endpoint runs in the HTTP server task, so panning works even while a live preview is open. The legacy auto-detect path still notifies the helper (`HUMAN`) for a gentle alert sweep.

Smooth, low-latency motion. The helper firmware (`firmware/helper_servo/`) uses a **non-blocking** stepper that eases the servo toward a target angle ($\sim 2^\circ$ every 8 ms) instead of a blocking sweep; incoming commands only update the target, so a new command takes effect immediately and the UART is never stalled. On the app side (`LiveMonitorScreen.js`) a dependency-free slider (React Native `PanResponder`) maps touches by absolute screen position (`pageX` relative to the measured track) rather than child-relative `locationX`, fixing a low-angle mis-read/glitch; a latest-wins sender streams the angle live while dragging without piling up requests, plus $\pm 15^\circ$ nudge, center, and preset buttons.

Live-preview reliability. The camera path was set to `CAMERA_GRAB_LATEST` with `CAMERA_FB_IN_PSRAM` so the freshest frame is always served (no “stuck” frame when panning), and the WebSocket JPEG quality was lowered (`40 → 25`) for smaller, more consistent frames over weak Wi-Fi. A brownout-detector disable (`RTC_CNTL_BROWN_OUT_REG`) was added as a band-aid against Wi-Fi current-spike resets during streaming, with the real fix noted as a proper 5 V supply and bulk capacitor.

Supporting assets. Added a standalone dual-ESP32 UART test rig under `firmware/tests/dual_esp_uart` (helper + main-stub + README documenting the boot-safe AI-Thinker pin set and wiring), and `tools/setup_mosquitto_for_eagleeye.ps1` to make the local Mosquitto broker reachable by the camera on the PC hotspot (listener on all interfaces, anonymous access, firewall rule) for end-to-end testing.

3 Suggested maintenance

- After each merged feature, append a dated subsection (one paragraph + bullet list if needed) and re-run `pdflatex`.
- Keep commit messages descriptive; this report is easier to maintain when subjects follow `type(scope): summary` convention.

4 Compilation

```
cd docs/daily_changelog
pdflatex EagleEye_Daily_Change_Report.tex
pdflatex EagleEye_Daily_Change_Report.tex
```